МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»
ФАКУЛЬТЕТ ІНФОРМАТИКИ ТА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ
КАФЕДРА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ

Лабораторна робота №2

з дисципліни «Алгоритми та структури даних»

на тему *«Зв'язані динамічні структури даних. Списки»*

Виконав:
студент групи ІМ-о51
Ушканов Т.А.

Перевірив:
ас. Череватенко О.В.

Київ 2026

Завдання

Мета роботи:

Метою лабораторної роботи №2 «Зв'язані динамічні структури даних. Списки» є засвоєння теоретичного матеріалу та набуття практичного досвіду використання зв'язаних динамічних структур даних у вигляді одно- та двозв'язних списків при складанні різних алгоритмів.

Варіант: 18

Ключами елементів списку є цілі ненульові числа, які розташовуються у наступному порядку: 10 додатних, 10 від'ємних і т. д. Кількість елементів списку n повинна бути кратною 20. Перекомпонувати список так, щоб розташування елементів було наступним: 5 додатних, 5 від'ємних і т. д., не використовуючи додаткових структур даних, крім простих змінних (тобто «на тому ж місці»).

Постановка задачі:

1. Створити список з n (n > 0) елементів (n вводиться з клавіатури), якщо інша кількість елементів не вказана у конкретному завданні за варіантом.
2. Тип ключів (інформаційних полів) задано за варіантом.
3. Вид списку (черга, стек, дек, прямий однозв'язний лінійний список, обернений однозв'язний лінійний список, двозв'язний лінійний список, однозв'язний кільцевий список, двозв'язний кільцевий список) вибрати самостійно з метою найбільш доцільного розв'язку поставленої за варіантом задачі.
4. Створити функції (або процедури) для роботи зі списком (для створення, обробки, додавання чи видалення елементів, виводу даних зі списку в консоль, звільнення пам'яті тощо).
5. Значення елементів списку взяти самостійно такими, щоб можна було продемонструвати коректність роботи алгоритму програми. Введення

значень елементів списку можна виконати довільним способом (випадкові числа, формування значень за формулою, введення з файлу чи з клавіатури).

6. Виконати над створеним списком дії, вказані за варіантом, та коректне звільнення пам'яті списку.

7. При виконанні заданих дій, виводі значень елементів та звільненні пам'яті списку вважати, що довжина списку (кількість елементів) невідома на момент виконання цих дій. Тобто, не дозволяється зберігати довжину списку як константу, змінну чи додаткове поле.

При проєктуванні програм слід врахувати наступне:

1) при виконанні завдання кількість операцій (зокрема, операцій читання й запису) має бути мінімізованою, а також максимально мають використовуватися властивості списків;

2) повторювані частини алгоритму необхідно оформити у вигляді процедур або функцій (для створення, обробки, виведення та звільнення пам'яті списків) з передачею списку за допомогою параметра(ів).

3) у таких видів списків, як черга, стек, дек функції для роботи зі списком мають забезпечувати роботу зі списком, відповідну тому чи іншому виду списку (наприклад, не можна додавати нові елементи всередину черги);

Текст програм/и:

```
using System;
using System.Collections.Generic;

public class CollectionChains
{
    public static LinkedList<int> BuildList(int n, bool p = true)
    {
        if (n <= 0)
        {
```

```
            throw new ArgumentOutOfRangeException(nameof(n), "Value of n must be
positive");
        }
        else if (n % 20 != 0)
        {
            throw new ArgumentOutOfRangeException(nameof(n), "Value of n must be divisible
by 20");
        }

        LinkedList<int> list = new LinkedList<int>();

        for(int i = 0; i < n; i++)
        {
            int x = i + 1;
            int y = p == true ? 1 : -1;

            list.AddLast(y);

            CheckForSwitch(x, 10, ref p);
        }

        return list;
    }

    public static void Rearrange(LinkedList<int> list, int steps = 5,
    int previousSteps = 10)
    {
        LinkedListNode<int> node = list.First;

        int i = 1;

        while (node != null)
        {
            LinkedListNode<int> nextNode = node.Next;

            if (i % (previousSteps * 2) == previousSteps + steps)
            {
                LinkedListNode<int> save = node;
                LinkedListNode<int> prev = save;

                for (int x = 0; x < previousSteps; x++)
                {
                    if (prev.Previous != null)
                        prev = prev.Previous;
                }

                for (int y = 0; y < steps; y++)
                {
                    LinkedListNode<int> toMove = save;
                    save = save.Previous;
                    list.Remove(toMove);
                    LinkedListNode<int> inserted = list.AddAfter(prev, toMove.Value);
                    prev = inserted;
                }
```

```
            }

            i++;
            node = nextNode;
        }
    }

    public static void FreeList(LinkedList<int> list) => list.Clear();
    public static void PrintList(LinkedList<int> list, int n = 0)
    {
        if (n > 0)
        {
            Console.WriteLine($"n = {n}");
        }

        LinkedListNode<int> node = list.First;
        int i = 1;
        while (node != null)
        {
            Console.WriteLine($"element {i}: {node.Value}");
            i++;
            node = node.Next;
        }
    }
    private static void CheckForSwitch(int x, int y, ref bool p)
    {
        if (x % y == 0)
        {
            p = SwitchBool(p);
        }
    }
    private static bool SwitchBool(bool x)
    {
        return !x;
    }
}
```

Тестування:

```
початкові дані:
n = 20
element 1: 1
element 2: 1
element 3: 1
element 4: 1
element 5: 1
element 6: 1
element 7: 1
element 8: 1
element 9: 1
element 10: 1
element 11: -1
element 12: -1
element 13: -1
element 14: -1
element 15: -1
element 16: -1
element 17: -1
element 18: -1
element 19: -1
element 20: -1
одержані дані:
n = 20
element 1: 1
element 2: 1
element 3: 1
element 4: 1
element 5: 1
element 6: -1
element 7: -1
element 8: -1
element 9: -1
element 10: -1
element 11: 1
element 12: 1
element 13: 1
element 14: 1
element 15: 1
element 16: -1
element 17: -1
element 18: -1
element 19: -1
element 20: -1
```

Висновки:

У лабораторній роботі №2 була опрацьована тема рекурсивних алгоритмів. Був реалізован рекурсивний алгоритм перекомпонування списку.